From Prediction to Persuasion: Agentic Recommendation Reason Generation for Regulatory-Compliant Financial AI

Seonkyu Jeong[1], Euncheol Sim[1], Youngchan Kim[1]

[1]*Independent Research*

Abstract. Financial product recommendation systems must explain *why* a product is recommended — first and foremost because persuasion requires narrative, not probability: a customer who asks "Why should I buy this fund?" needs a reason they can accept, not a score they must trust. Regulators (Korean FSS, EU AI Act) are one important audience for such explanations, but the primary driver is the human act of persuasion itself. We present a multi-stage pipeline that bridges the gap between model prediction and human persuasion: (1) IG-guided knowledge distillation from a heterogeneous-expert PLE teacher to per-task LGBM students, preserving explanation-relevant features while enabling GPU-free CPU inference; (2) a multi-agent recommendation reason generation pipeline where three specialized serving agents (Feature Selector, Reason Generator, Safety Gate) collaboratively produce natural-language explanations grounded in business-mapped feature attributions; (3) two operational agents (OpsAgent and AuditAgent) that interpret monitoring outputs and compliance reports in natural language, enabling regulation-compliant MLOps for small teams without dedicated MLOps staff; (4) regulatory compliance by design, with built-in drift monitoring, fairness auditing, and governance reporting aligned to Korean FSS guidelines, the EU AI Act, and the Korean AI Basic Act. We evaluate distillation quality (AUC gap $<$ X%), reason generation quality via human evaluation (N domain experts, Y% preference over template baselines), and Safety Gate reliability (precision/recall). The system achieves sub-100ms serving latency on AWS Lambda at a fraction of the cost of dedicated GPU inference servers.

Keywords: Recommendation explanation, Knowledge distillation, LLM agents, Regulatory compliance, Financial AI, EU AI Act

1 Introduction

1.1 The Persuasion Problem

The final deliverable of a financial recommendation system is not a probability but a *reason the customer can accept*. A model outputting $P(\text{invest} \mid x) = 0.73$ provides no value to:

- The **customer** who asks "Why should I buy this fund?"
- The **relationship manager** who needs a talking point for the sales call.
- The **regulator** who demands "Why was this decision made?" under Article 13 of the EU AI Act.

Existing explanation approaches are insufficient:

- **SHAP/LIME**: "feature_237 contributed 0.12" — no business meaning, unstable under perturbation [9, 13].
- **Template-based**: "Customers like you also bought X" — rigid, ignores individual context.
- **Direct LLM generation**: Hallucination risk, no grounding in actual model reasoning, regulatory non-compliance.

1.2 Our Approach

We propose a full-chain solution from prediction to persuasion:

1. **Knowledge Distillation**: A heterogeneous-expert PLE teacher (14 tasks, 7 experts, 350 features; organized by the companion paper's reductionist two-axis framework of Financial DNA $\times$ Data Modality) is distilled into per-task LGBM students using IG-guided feature selection. The teacher employs sigmoid CGC gates (inspired by [11]) instead of standard softmax gates, enabling independent expert contribution without harmful inter-expert competition — a critical improvement for heterogeneous expert architectures (detailed in companion paper). This enables GPU-free serving while preserving the features that matter for explanation.

2. **Multi-Agent Reason Generation**: Three specialized LLM agents collaborate in a pipeline — Feature Selector chooses explanation-worthy features, Reason Generator produces natural-language narratives, and Safety Gate validates regulatory compliance.

3. **Compliance by Design**: Drift monitoring, fairness auditing, audit trails, and governance reporting are

embedded in the pipeline, not bolted on after deployment.

1.3 Contributions

1. **IG-guided Distillation with Explanation Preservation**: Feature selection optimizes not only for prediction accuracy but also for explanation material availability.
2. **Feature Business Reverse-Mapping**: A systematic registry (`interpretation_registry`) that maps every feature to a business-interpretable description, enabling grounded explanation generation.
3. **3-Agent Reason Generation Pipeline**: Role-separated agents (selection → generation → safety) with independent improvement and audit logging.
4. **Safety Gate for Financial Compliance**: Automated checking for hallucination, inappropriate investment advice, and regulatory violations (Financial Consumer Protection Act (금소법), Suitability Principle (적합성 원칙)).
5. **5-Agent Architecture (3 Serving + 2 Ops)**: Beyond the 3 serving agents, two operational agents (OpsAgent, AuditAgent) interpret monitoring and compliance outputs in natural language, enabling small-team MLOps without dedicated MLOps staff.
6. **Regulatory Compliance Architecture**: Explicit mapping of system components to Korean FSS guidelines, EU AI Act articles, and the Korean AI Basic Act.
7. **Human Evaluation Protocol**: Systematic evaluation of recommendation reason quality by domain experts.

2 Related Work

2.1 Knowledge Distillation for Recommendation

Hinton et al. [6] established the teacher-student paradigm via temperature-scaled soft labels. In recommendation, distillation has been applied to compress deep models into lightweight servers. CKD (Collaborative Knowledge Distillation) and ranking distillation preserve recommendation-specific structure.

Gap: No prior work selects distillation features based on *explanation value* alongside predictive value.

2.2 Recommendation Explanation Generation

Recommendation explanation approaches fall into three categories:

Template-based methods map model outputs to prewritten phrases ("Recommended because you recently viewed similar products"). Amazon and Netflix use variants of this approach at scale. Templates are safe and fast but lack personalization and flexibility — the same template serves millions of different customer contexts.

Attention-based methods use model-internal attention weights as feature attributions for explanation. While more dynamic than templates, attention weights are contested as reliable explanations, and the resulting attributions still require translation into business language.

LLM-based generation is an emerging approach that uses large language models to produce natural-language explanations from model outputs. Recent work explores LLM-augmented recommendation explanation, but faces challenges of hallucination, regulatory compliance, and grounding in actual model reasoning.

Gap: No system combines structural model explainability (gate weights) with multi-agent LLM-based generation under regulatory safety constraints, where each agent has a specialized role (selection, generation, validation) and all outputs are audit-logged.

2.3 Responsible AI in Finance

Korean FSS published AI guidelines (2021) [4] and model risk management directives requiring explainability, fairness monitoring, and audit trails. The EU AI Act [2] classifies financial credit/recommendation as high-risk AI, mandating transparency (Art. 13), human oversight (Art. 14), and accuracy (Art. 15). The EBA [1] calls for "interpretable" models in internal risk assessments. Korea's AI Basic Act [10] (passed December 2024, promulgated January 2025) adds domestic high-impact AI classification.

Pearl [12] argues that true explanation requires causal understanding, not mere statistical association — a position increasingly echoed by financial regulators who demand explanations reflecting the actual decision mechanism [14].

Gap: No recommendation system provides an explicit, verifiable mapping from regulatory requirements to system architecture components, with explanations grounded in causal reasoning rather than post-hoc correlation.

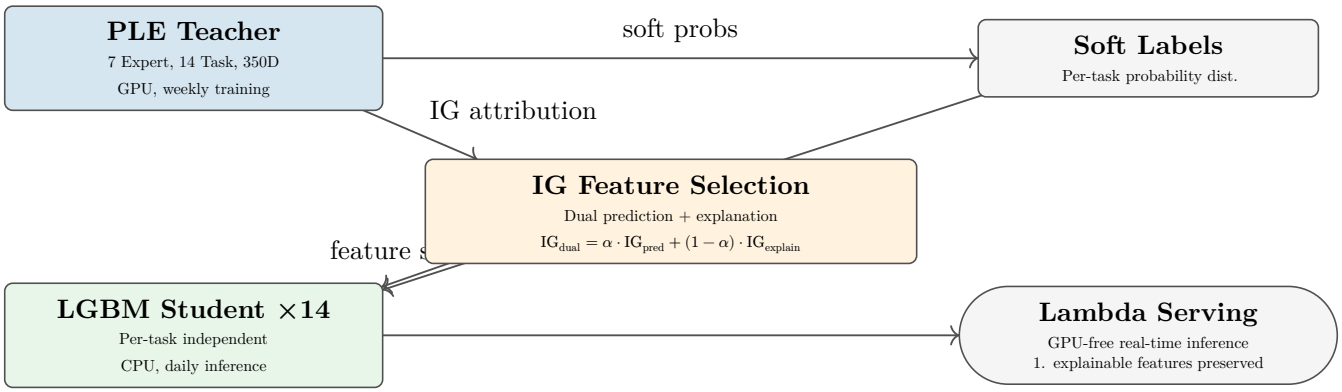

Figure 1: Teacher-student distillation architecture. PLE teacher's soft labels and IG-based feature selection distill into per-task LGBM student models.

2.4 LLM Safety and Grounding

Deploying LLMs for customer-facing financial text introduces specific risks: hallucination (stating non-existent product features), inappropriate advice (recommending unsuitable products for the customer's risk profile), and regulatory violation (breaching 금소법 or 적합성 원칙).

Retrieval-Augmented Generation (RAG) mitigates hallucination by grounding generation in retrieved factual context. Our approach extends this: rather than retrieving from a general knowledge base, we ground generation in *model-internal feature attributions* that have been reverse-mapped to business descriptions. This ensures the generated text reflects what the model actually computed, not what an LLM independently "knows."

Content filtering and safety gates provide a final defense layer. Our 3-agent architecture separates generation from validation, enabling independent improvement of each component.

3 Knowledge Distillation

3.1 Teacher-Student Architecture

The teacher model (PLE with 7 heterogeneous experts, 14 tasks, 350 features; see companion paper for architecture details) produces soft probability outputs that serve as training targets for per-task LGBM [7] students. The teacher's value as a distillation source stems from its *structural guarantee* against *expert collapse*: because the seven experts are architecturally distinct (DeepFM, Mamba, HGCN, PersLay, etc.), they cannot converge to the same function, ensuring the soft labels encode genuinely multi-faceted customer understanding. Note on HGCN: the graph expert receives 27-dimensional `merchant_hierarchy` features (MCC L1 → L2 → code Poincaré embeddings), not product co-holding features. This distinction matters for distillation: IG scores from the HGCN expert rank highly for MCC-dependent tasks (e.g., top_mcc_shift), and the preserved HGCN features produce explanation-grounded statements such as "customer shows sustained preference for merchant category X" rather than generic co-holding signals.

The key design decision is *per-task distillation*: rather than a single student model for all 14 tasks, we train 14 independent LGBM models, each learning one task's soft labels. This enables: (1) per-task feature selection (different tasks benefit from different features), (2) independent retraining (if one task drifts, only its student is re-distilled), (3) interpretable feature importance per task (LGBM's built-in feature importance aligns with the business reverse-mapping for explanation generation).

Four tasks from the original 18-task PLE teacher were excluded from the distillation pipeline: income tier, tenure stage, spend level, and engagement score. These tasks represent deterministic feature transformations — for instance, income tier is simply a quantile bucket of the raw income feature, which is already a model input. A student model can perfectly reconstruct such labels from its input features, making the distillation trivially solvable and uninformative. The remaining 14 tasks represent genuine prediction objectives where the label cannot be deterministically derived from input features.

Lifecycle separation:

- **Teacher**: retrained weekly/monthly on SageMaker (GPU required, comprehensive). The teacher captures complex inter-task relationships via adaTT and non-linear expert interactions that LGBM cannot directly learn.
- **Students**: re-distilled daily with fresh soft labels (CPU only, fast). Daily re-distillation tracks data drift without the cost of GPU training.
- **Champion-Challenger**: automatic comparison of new student vs. current production model. If the new student's AUC drops below threshold, the update is blocked and an alert is raised.

This architecture resolves a fundamental tension in financial AI: the *model that learns best* (deep PLE with GPU) is not the *model that serves best* (lightweight LGBM on CPU Lambda). Knowledge distillation [6] bridges this gap, and the FD-TVS (Friedman Decomposition–Transitory Variance Scoring) system [5] further weights the student's predictions by income stability type. FD-TVS applies Friedman's Permanent Income Hypothesis to decompose each customer's observed income into permanent (stable, long-term) and transitory (bonus, irregular) components via HP or Kalman filtering. Customers whose income is predominantly transitory receive lower confidence weights for long-horizon product recommendations, preventing the system from recommending products that assume stable cash flow to customers with volatile income patterns.

Temperature $T = 1$ for tree-based students. The original distillation formulation [6] raises the softmax temperature T to soften the teacher's output distribution, amplifying the relative probabilities of non-target classes ("dark knowledge") so that the student neural network receives gradient signal across all classes. However, this rationale does not transfer to LGBM students: (1) LGBM learns via split-based optimization, not back-propagation, so vanishing gradients on tail classes are not an issue; (2) raising T compresses the probability range — for a binary task with 2.98% positive rate, $T = 5$ maps teacher output from $[0.03, 0.97]$ to $[0.48, 0.52]$, effectively destroying the class-discriminative signal; (3) the Soft GBM study [3] shows that hard CART trees already struggle to learn multi-dimensional soft label vectors, and flattening these further via high T only exacerbates the problem. We therefore set $T = 1$, using the teacher's calibrated probabilities as-is and relying on the custom LGBM objective function to inject the soft-label gradient at each split. This design choice is specific to tree-based student models; should a neural student architecture be adopted in the future, temperature scaling can be re-introduced at that point.

3.2 IG-based Feature Selection

Integrated Gradients [15] computes per-feature attribution from the teacher model. We select top-k features ranked by IG importance, with a dual objective:

$$\text{score}(f) = \alpha \cdot \text{IG}_{\text{pred}(f)} + (1 - \alpha) \cdot \text{IG}_{\text{explain}(f)}$$

where IG_{pred} measures predictive contribution and $\text{IG}_{\text{explain}}$ measures the feature's value as explanation material.

The explanation score $\text{IG}_{\text{explain}(f)}$ is derived from the feature's reverse-mapping richness in the interpretation registry: features with detailed business descriptions, clear directionality, and natural-language templates receive higher scores. For example, `hmm_lifecycle_prob_growing` (explanation-rich: "customer is in growth stage") scores higher than `mamba_temporal_d17` (explanation-poor: generic embedding dimension).

This dual-objective selection ensures that the student model retains not only the most predictive features but also the features that generate the most compelling recommendation reasons. The hyperparameter α controls the trade-off: $\alpha = 1$ optimizes purely for prediction, $\alpha = 0$ purely for explanation quality. We empirically find $\alpha = 0.7$ balances both objectives.

The resulting feature set is typically 40–80 features per task (down from 350), achieving >95% of teacher AUC while providing sufficient explanation vocabulary. Critically, the dual-objective selection preserves features across all four Financial DNA dimensions (engagement, lifecycle, value, consumption) and multiple Data Modalities (temporal, graph, topological, etc.) from the companion paper's two-axis decomposition, ensuring the student retains multi-faceted understanding rather than collapsing to a single dominant feature group.

3.3 Distillation Results

Benchmark v4 produces meaningful label distributions across all 14 tasks. Notably, `nba_primary` reaches 60% no-NBA (down from 91% in earlier benchmarks) and `top_mcc_shift` reaches 50% positive rate (down from 92%), removing the near-degenerate class imbalance that previously made teacher-student fidelity numbers artificially easy to achieve. These distributions make the distillation gaps below genuinely informative.

Teacher-student fidelity metrics are reported per task type: AUC gap for binary classification tasks, F1-macro gap for multiclass tasks, and MAE gap for regression tasks. This avoids conflating metrics with incompatible semantics across task types.

The `interpretation_registry` serves as the **data contract** between distillation and reason generation. Distillation (Section 3) decides *which* features to preserve in the student model — optimizing for both predictive accuracy and explanation material via the dual-objective IG score. Reason generation (Section 4) decides *how* to explain those features to humans — mapping each preserved feature to business-interpretable language. This separation means the two stages can evolve independently: improving feature selection does not require rewriting explanation templates, and vice versa.

Task	Teacher AUC	Student AUC	Gap
has_nba	–	–	–
churn_signal	–	–	–
will_acquire_deposits	–	–	–
will_acquire_investments	–	–	–
…	…	…	…

Table 1: Distillation results per task (TODO). Binary tasks: AUC gap; multiclass: F1-macro gap; regression: MAE gap.

Feature	Business Mapping	Explanation Template
hmm_lifecycle _prob_growing	Growth stage probability	"Your asset portfolio is in a growth phase"
mamba_temporal _d3	3-month spending trend	"Your spending has been increasing recently"
hgcn_hierarchy _d5	Product category position	"Investment products are a natural next step"
synth_stability	Transaction stability	"You maintain a stable transaction pattern"
gmm_cluster _prob_3	Segment probability	"You share characteristics with active investors"

Table 2: Feature reverse-mapping examples.

4 Recommendation Reason Generation

4.1 Feature Business Reverse-Mapping

Every feature in the system is registered in an `interpretation_registry` with structured business metadata:

This registry serves dual purposes: (1) grounding material for the Reason Generator agent, and (2) audit trail showing which features influenced each recommendation.

`InterpretationRegistry` interprets features into Korean via a 5-level cascade: Level IG (IG sign direction + task context) → Level 3 (feature×task manual overrides) → Level 2 (group×task) → Level 1 (group×task_group auto-generated) → Level RM (`ReverseMapper` glossary templates). Only features unresolved by this cascade fall to raw fallback. `ReverseMapper` is integrated as Level RM, so glossary value-substitution templates (e.g., "월 평균 {value}건 거래") operate as part of the cascade. All fallback text outputs Korean.

4.2 3-Agent Pipeline Architecture

4.2.1 Agent 1: Feature Selector

Selects features for explanation based on:

- IG attribution scores (model-driven relevance)
- Business reverse-mapping richness (explanation-driven relevance)
- Customer context (personalization: different features matter for different customer profiles)

The Feature Selector uses the Financial DNA axis from the companion paper's two-axis framework to select features from the relevant customer dimension (e.g., lifecycle features for a churn recommendation, value features for an investment recommendation), ensuring explanations address the dimension most pertinent to the recommended action.

4.2.2 Agent 2: Reason Generator

Receives selected features with their business reverse-mappings and generates a natural-language recommendation reason. The Reason Generator structures its narrative around Financial DNA groups — opening with the customer's current state (lifecycle), grounding the recommendation in observed behavior (engagement/consumption), and framing the value proposition in terms the customer recognizes (value dimension) — producing what the companion paper calls *multi-faceted customer understanding* rendered as persuasive text. Grounding constraints:

- Must reference only features actually present in the selection.
- Must use business terminology appropriate for the target audience (customer vs. relationship manager).

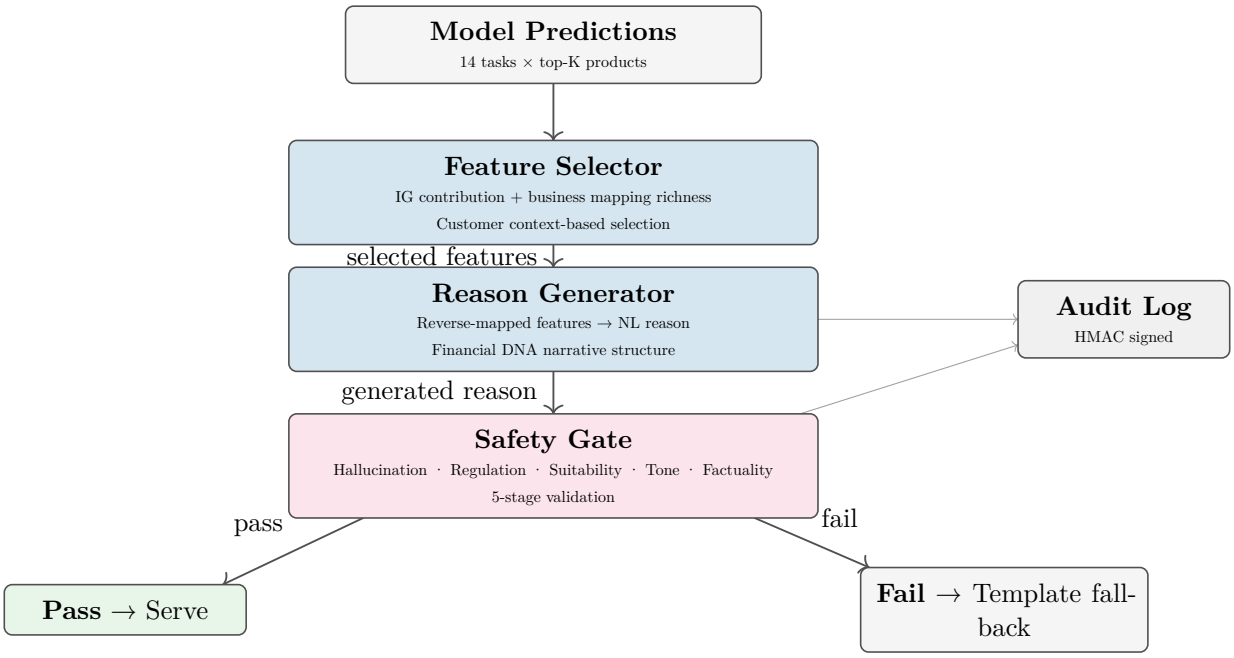

Figure 2: 3-agent recommendation reason generation pipeline. Feature Selector → Reason Generator → Safety Gate.

Check Category	Criteria
Hallucination	No claims about facts not in feature data
Regulatory	No violation of 금소법, 적합성 원칙
Appropriateness	No unsuitable investment advice for risk profile
Tone	Professional financial advisory language
Factual	Numerical claims match actual feature values

Table 3: Safety Gate validation criteria.

- Must follow financial domain tone and compliance guidelines.

Example output (customer-facing):
"You are currently in an asset growth phase, with spending trending upward over the past three months. Given your product portfolio structure, an investment fund is a natural next step. Considering your stable transaction pattern, we recommend a balanced moderate-risk fund."

Figure 3: Generated recommendation reason example (translated from Korean).

4.2.3 Agent 3: Safety Gate

Validates the generated reason against:

On failure: automatic fallback to template-based safe reason. All gate decisions are logged for audit trail. Upstream of the 3-agent pipeline, the `ConstraintAwareEngine` applies eligibility and suitability filters — verifying investment experience, risk tolerance, and product-specific constraints — so that no recommendation reaches the customer without passing a suitability check, as required by the Korean Financial Consumer Protection Act (금소법) Article 19 (적합성 원칙).

4.2.4 Serving Model Selection

Customer-facing recommendation reasons require natural, professional Korean text. The optimal model differs by deployment environment: **On-premises (air-gapped)**: Exaone 3.5 7.8B (LG AI Research, Apache 2.0) — Korean-specialized training produces more natural financial honorific tone than same-class models (Llama, Qwen). Runs on RTX 4070 12GB. **Cloud (AWS)**: L2a rewriting uses Solar Pro 22B (Upstage, Bedrock Marketplace) — top performance on Korean benchmarks (KMMLU). L2b self-critique also uses Solar (generator $\leq$ critic model principle). SelfChecker factuality scoring uses Claude Haiku. Bedrock ensures that input/output data is never transmitted to model providers (Anthropic, Upstage) and is never used for model training. VPC PrivateLink enables invocation without traversing the public internet, ensuring that financial customer data never leaves the AWS Region (ap-northeast-2) — structurally satisfying the data gov-

ernance requirements of Korean FSS AI guidelines and the Personal Information Protection Act.

4.2.5 Fact Compression Layer (Mem0 Adoption)

While `InterpretationRegistry` provides feature-level interpretation, `FactExtractor` adds **customer-level narrative facts**. A rule-based engine extracts Korean facts like "예적금 중심 포트폴리오" (deposit-focused portfolio), "최근 3개월 펀드 관심 증가" (recent fund interest growth), "리스크 회피 성향" (risk-averse tendency) from feature values — deterministically and without any LLM calls.

These facts are extracted at Phase 0 batch time, stored in `ContextVectorStore`, and injected into the L2a prompt as a "Customer Facts" section at serving time. Solar Pro then generates reasons **with customer understanding**, not just raw feature values.

Rules are defined in `configs/financial/fact_extraction.yaml` (15 categories covering portfolio composition, interests, risk tolerance, lifecycle, etc.) and new facts can be added with config-only changes.

4.3 Caching Strategy and Asynchronous L2a Architecture

Recommendation reasons are served via a 3-layer asynchronous architecture:

1. **L1 (Template)**: returned immediately on customer request. No LLM call. `TemplateEngine` generates deterministic Korean reasons based on IG top-K feature business reverse-mappings. Features pass through `InterpretationRegistry`'s 5-level cascade (IG direction → L3 → L2 → L1 → ReverseMapper) to produce enriched 3-tuples `(feature_name, IG_value, Korean_interpretation)`.
2. **L2a (LLM Rewrite)**: submitted asynchronously via SQS. Solar Pro refines L1 reasons into natural Korean. Results are cached in DynamoDB for subsequent requests. VIP customers receive priority processing.
3. **L2b (Quality Validation)**: applies a 5-stage safety gate to L2a output — (1) PromptSanitizer, (2) PII detection (Korean resident registration number, card numbers, etc.), (3) SelfChecker (compliance + injection + factuality), (4) grounding verification (number cross-check), (5) 5% human review sampling. Pass promotes to L2b; failure falls back to L1.

Caching uses a dual backend (in-memory + DynamoDB) with composite key `customer_id + product_id + task_name` and TTL-based auto-expiry. Of 941K customers, L2a targets (5% sample, 47K items) are processed by 5 parallel Solar workers in 8 minutes at $0.10 cost.

5 Operational Agent Pipeline

The preceding section described three serving agents that generate recommendation reasons on the customer-facing path. This section introduces two additional agents that operate on a separate, batch-only path: interpreting monitoring outputs and compliance reports so that a small operations team can maintain regulation-compliant MLOps without dedicated MLOps staff.

5.1 Motivation: Human–AI Role Separation

Not all operational tasks require the same cognitive mode. Formalized, repetitive monitoring — checking whether PSI exceeded a threshold, whether fairness metrics dipped below a limit, whether gate entropy shifted — is well-suited to LLM agents that run on a batch schedule (daily or weekly). Non-routine judgment — diagnosing data contamination, interpreting business context shifts, responding to regulatory interpretation changes — requires human expertise that no current LLM can reliably provide.

The key design insight is role separation: agents tell *what changed*; humans decide *why it changed and what to do*. This division makes regulation-compliant operations feasible for small teams (1–2 people) without dedicated MLOps staff, because the agents eliminate "dashboard fatigue" — instead of checking 10 metrics daily, the human reads one natural-language summary.

5.2 Two Operational Agents

5.2.1 OpsAgent (Operations Agent)

The OpsAgent runs after training completion and drift monitoring DAG executions.

Inputs: `eval_metrics.json`, training logs, CGC gate entropy, PSI drift reports.

Output: A "Model Health Report" in natural language.

> **Example OpsAgent output:**
> "Task 'churn_signal' AUC dropped 3.2% vs. previous version. Gate entropy for LightGCN expert decreased to 0.4 (from 1.2), suggesting routing concentration. Recommend examining recent data distribution changes before promotion."

Figure 4: OpsAgent Model Health Report example.

Triggers: drift monitoring DAG completion, training job completion.

Principle	Rationale
Batch-only, never real-time	No serving path dependency; agents run asynchronously after DAG completion
Per-task optimal model assignment	Reason generation: Solar Pro (Korean-specialized). Agent dialog/consensus: Claude Sonnet (contextual reasoning). Judgment: Claude Haiku (low cost). Embeddings: Titan V2. On-prem: Exaone 3.5 (reasons) + Qwen 2.5 14B Q4 (consensus)
Reports deposited to shared folder	Alerts via Slack/email only on anomalies; human reviews at their own pace
Agent outputs are audit artifacts	Immutable, HMAC-signed; the report itself is evidence of monitoring
Cost: \$0.03/day (3× consensus)	1–2 small-model calls with structured input per execution cycle

Table 4: Operational agent design principles.

The OpsAgent does not make promotion decisions — it surfaces the information a human needs to make one. When all metrics are within normal bounds, the report is brief ("All 14 tasks within tolerance; no action required"). When anomalies are detected, the report identifies *which* tasks and *which* experts are affected, providing actionable context rather than raw numbers.

5.2.2 AuditAgent (Audit/Compliance Agent)

The AuditAgent runs after fairness monitoring and governance DAG executions.

Inputs: `FairnessMonitor` reports (DI/SPD/EOD), audit trail integrity checks, opt-out statistics, governance checklist status.

Output: A "Regulatory Compliance Report" in natural language.

Example AuditAgent output:
"Disparate Impact for age_group 60+ on lending recommendations: DI = 0.73 (threshold: 0.80). 3 consecutive days below threshold. Recommend human review per FSS AI RMF requirement G-3."

Figure 5: AuditAgent Regulatory Compliance Report example.

Triggers: fairness monitoring DAG completion, quarterly governance cycle.

The AuditAgent converts quantitative fairness metrics into regulatory language, explicitly referencing the applicable regulation (FSS guideline number, EU AI Act article) so that the human reviewer can act without cross-referencing documentation.

5.3 Design Principles

The critical constraint is that operational agents have *no shared state* with the serving path. The serving pipeline (L1 Feature Selector → L2a Reason Generator → L2b Safety Gate) produces customer-facing outputs; the operational pipeline (DAG completion → OpsAgent/AuditAgent → report storage → human review) produces internal operations artifacts. This separation ensures that an operational agent failure can never degrade customer-facing service.

5.4 Model Selection for Operational Agents

Unlike serving agents, which require Korean-language fluency for customer-facing text, operational agents process structured JSON inputs and produce logical assessments — natural language fluency is secondary to reasoning accuracy. **On-premises (air-gapped)**: Exaone 3.5 7.8B (Korean reason generation) + Qwen 2.5 14B Q4 (agent consensus). Sequential loading on RTX 4070 12GB VRAM. **Cloud (AWS)**: per-task optimal models — Solar Pro (Korean L2a reason generation/critique), Claude Sonnet (agent dialog, 3-agent consensus), Claude Haiku (SelfChecker factuality judgment), Claude Opus (quarterly deep audit), Titan Embeddings V2 (vectorization). The Bedrock infrastructure is shared between reason generation and agents; quota competition is resolved via time-slot separation. In both deployments, operational agents execute only 1–2 calls per DAG cycle, keeping cost and latency negligible regardless of model choice.

5.5 Practical Value

The operational agents address three concrete pain points in financial AI operations:

1. **Dashboard fatigue elimination**: Instead of monitoring 10+ metrics dashboards daily, the operations team reads one natural-language summary — "check when you come in to work."
2. **Automatic audit evidence accumulation**: When regulators ask "How did you respond to drift event X?", the institution produces the OpsAgent's report

from that date plus the human's subsequent action record, forming a complete incident response trail.

3. **Small-team MLOps enablement**: The architecture makes regulation-compliant operations feasible without a large dedicated MLOps team. The agents handle the formalized interpretation; humans contribute the judgment that regulations actually require.

5.6 Pipeline Part Classification and Inspection Checklist

For systematic inspection, the pipeline is divided into six parts with 48 checklist items: P1 (Ingestion), P2 (Feature Engineering), P3 (Training/Distillation), P4 (Serving/Recommendation), P5 (Reason Generation), P6 (Monitoring/Governance). Each item is defined in YAML config with tool name, threshold, and verdict logic; OpsAgent handles 23 items, AuditAgent handles 25 items.

5.7 Tool Calling Architecture

38 tools (29 Query + 9 Action) are defined via JSON Schema, wrapping existing monitoring components (`DriftDetector`, `FairnessMonitor`, `SelfChecker`, etc.) as callable tools for the agents. Query tools can be called freely, while Action tools (incident creation, audit logging) require explicit approval, structurally enforcing the Query/Action boundary.

5.8 3-Agent Consensus Mechanism

To structurally mitigate hallucination risk in LLM-based interpretation, three independent Sonnet sessions run in parallel. Each agent is assigned a different perspective: α (conservative), β (statistical significance), γ (business impact).

Results are classified into three tiers: (1) **Consensus** (3/3): confirmed verdict, (2) **Majority** (2/3): priority review — immediate operator attention, (3) **Minority Report** (1/3): secondary review — dissenting opinion preserved separately.

The core principle is **minority report preservation**. Once identified, a minority opinion is locked and cannot be deleted — in operations/audit contexts, missing a signal is far more dangerous than a false alarm. Novel problem types are often caught first by the dissenting perspective while the majority, anchored to familiar patterns, overlooks them.

5.9 Diagnostic Case Store

Inspection reports are not disposable artifacts but accumulate as an operational knowledge base. A LanceDB-based `DiagnosticCaseStore` stores structured metadata (part, item, verdict, severity) and text embeddings (finding + cause + action) for three purposes: (1) **similar case search**: referencing past response history when a new anomaly occurs, (2) **statistical analysis**: aggregating frequent WARN types per part, mean resolution time, (3) **resolution tracking**: quantifying actual effectiveness of responses via (problem, action, post-action verdict) triples.

The case schema includes a `consensus_type` field to track the rate at which minority opinions turned out to be correct. This data serves directly as "continuous improvement evidence" for regulatory audits.

5.10 Selective Memory Framework Adoption

Core patterns from several 2026 agent memory frameworks (Mem0, Zep/Graphiti, Letta, SuperLocalMemory) were **selectively adopted** as incremental additions to existing infrastructure.

Adoption 1 — Temporal Knowledge Graph (Zep/Graphiti): `TemporalFactStore` uses a `(entity, attribute, value, valid_from, valid_to)` schema for audit evidence. Point-in-time queries like "What was customer A's state at 2026-03-15?" resolve as single filters. Shares the same LanceDB backend as `DiagnosticCaseStore` — zero new dependencies.

Adoption 2 — Mathematical Decay (SuperLocalMemory): `DiagnosticCaseStore.search_similar()` now applies $\exp\left(-\frac{\text{age}}{\tau}\right)$ weighting with a 90-day half-life default. **Original cases are preserved** — only search weights are adjusted, maintaining audit traceability.

Adoption 3 — Dialog Recall Memory (Letta): `DialogRecallMemory` stores past operator conversations in DynamoDB so that `BedrockDialogSession` can recall "that issue we discussed last week" across sessions.

Not adopted: LangMem's prompt self-improvement (audit risk — cannot answer "who approved this prompt?").

All adoptions are **opt-in** and do not affect existing behavior when not configured.

5.11 Change Detection and Impact Review

Changes to code, configuration, models, and data sources are detected via two channels: push (git hooks, `_PipelineState` callbacks, ingestion completion events) for immediate detection, and pull (manifest diff, serving metric polling) for periodic detection. When a change is detected, the checklist for affected pipeline parts is re-executed. In the AWS environment, Sonnet reads the diff

FSS Requirement	System Component	Verification
Explainability	Gate weights + 3-agent reason	Per-recommendation audit log
Fairness	FairnessMonitor (DI/SPD/EOD)	Weekly automated report
Model validation	Champion-Challenger	Pre-deployment comparison
Monitoring	DriftDetector (PSI)	Continuous, 3-day trigger
Audit trail	HMAC hash-chain logs	Immutable, 7 audit tables
Fallback	Template reason + kill switch	Instant manual override
Model risk mgmt	ModelCompetitionManager (champion-challenger)	Independent validation, manual approval
Customer suitability	ConstraintAwareEngine + eligibility filters	Pre-recommendation suitability check

Table 5: Korean FSS guideline compliance mapping.

Article	Requirement	System Component
Art. 13	Transparency	3-agent reason generation + feature attribution
Art. 14	Human oversight	Human-in-the-Loop review + kill switch
Art. 15	Accuracy & robustness	Ablation-validated degradation + drift monitoring
Art. 9	Risk management	MRM lifecycle: develop → validate → approve → monitor → retrain
GDPR Art. 22	Right to opt-out	Consent/opt-out audit table + manual override

Table 6: EU AI Act article-level compliance mapping.

and reasons about downstream impact, enabling operators to discuss the impact assessment interactively.

6 Regulatory Compliance

6.1 Korean FSS Guidelines Mapping

Table 5 maps the key requirements of Korean FSS AI guidelines to the corresponding system components and verification methods.

6.2 EU AI Act Mapping

Table 6 maps the core EU AI Act provisions to system compliance mechanisms. Financial recommendation systems are classified as high-risk AI under Annex III Section 5, requiring compliance with Title III Chapter 2 obligations.

6.3 Korean AI Basic Act (passed 2024.12, effective 2026.1)

Korea's AI Basic Act (passed by the National Assembly December 2024, promulgated January 2025, effective January 2026) [10] introduces a domestic high-risk AI classification framework. Financial product recommendation falls within the high-risk category, requiring impact assessment, transparency obligations, and human oversight. Our system's existing compliance architecture (drift monitoring, fairness auditing, audit trails, and human-in-the-loop review) aligns with the Act's requirements, with the governance reporting module generating documentation suitable for regulatory submission.

6.4 Monitoring and Governance

6.4.1 Human-in-the-Loop

Regulatory bodies (Korean FSS, EU AI Act Art. 14) require human oversight. The system implements this at multiple levels:

- **Reason sampling review**: Periodic human review of generated reasons.
- **Model replacement approval**: Champion-Challenger results require human sign-off.
- **Incident escalation**: Automated anomaly detection triggers human investigation.
- **Fairness review**: Periodic human audit of fairness metrics.

6.5 Pipeline Auditability for High-Risk AI

The preceding sections map individual system components to regulatory articles. However, the EU AI Act's high-risk AI requirements (Title III, Chapter 2) demand auditability of the *entire ML pipeline* — not just the

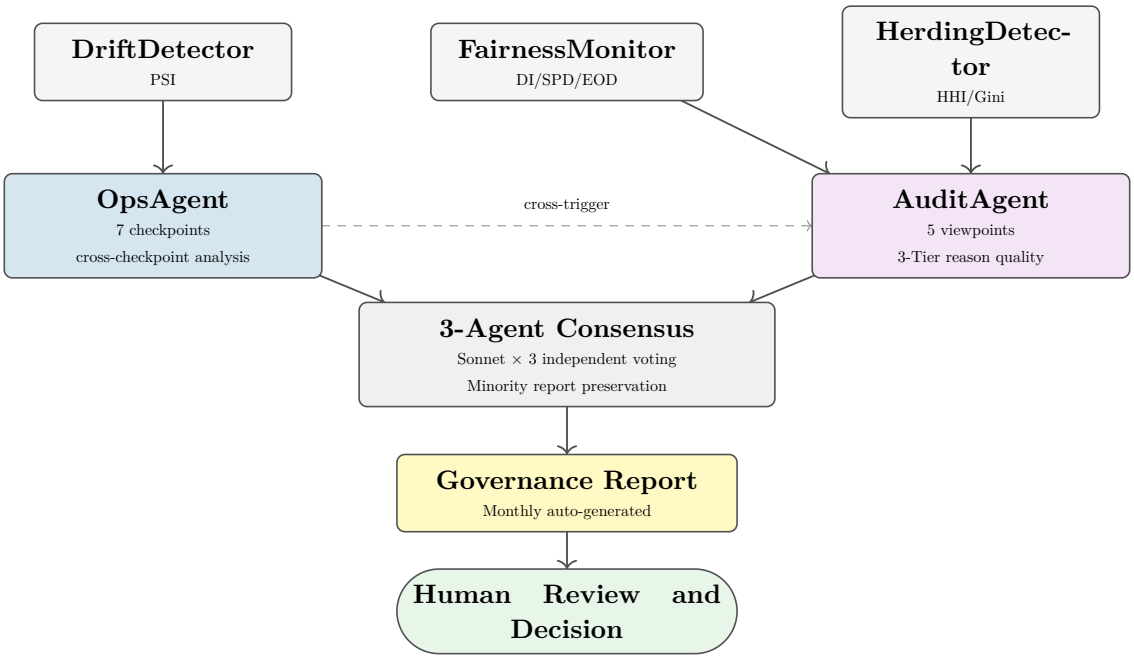

Figure 6: Monitoring and governance architecture. Monitoring components → Ops/Audit agents → Consensus → Governance report → Human review.

serving layer. This section demonstrates how our training infrastructure, described in the companion paper, satisfies these requirements end-to-end.

6.5.1 Data Governance (Art. 10)

Article 10 requires that training data be "relevant, representative, free of errors and complete" with appropriate data governance measures. Our Phase 0 pipeline (preprocessing → feature generation → label derivation → normalization → tensor storage) produces two audit artifacts at every run: `feature_stats.json` records per-column NaN ratios, zero-variance flags, distribution statistics, and generated feature counts; `label_stats.json` records class balance and positive rates for all 14 tasks. These artifacts constitute a verifiable data quality record that an external auditor can inspect without executing any code.

Reproducibility is guaranteed by config-driven processing: the entire pipeline is controlled by two YAML files (`pipeline.yaml` and `feature_groups.yaml`), so identical configs produce identical outputs given the same input data. No dataset-specific logic resides in executable code.

As a pre-training gate, `LeakageValidator` verifies that (1) the scaler was fit on training data only, (2) a temporal gap separates train/validation/test splits, and (3) the final sequence timestep does not overlap with label windows. Training is blocked if any check fails — ensuring data governance violations are caught *before* compute is consumed.

6.5.2 Technical Documentation (Art. 11)

Article 11 mandates technical documentation sufficient to assess compliance. Our system achieves this through config-as-documentation: the two YAML files fully specify feature groups, generator parameters, normalization stages, task definitions, loss weights, and expert routing. A config snapshot is saved alongside every training run, and `eval_metrics.json` captures the full training provenance (hyperparameters, data splits, final metrics, timestamps). Combined with fixed random seeds, any historical experiment can be exactly reproduced from its archived config.

6.5.3 Logging and Traceability (Art. 12)

Article 12 requires automatic recording of events for the identification of risks and substantial modifications. The training loop produces per-epoch logs with task-level loss breakdowns, enabling auditors to trace exactly when and where performance changed. NaN/Inf detection identifies *which task* produced anomalous gradients, rather than reporting a generic training failure. Gradient norm monitoring flags when norms exceed the clipping threshold by an order of magnitude, providing a training stability audit trail. Checkpoints are versioned with their corresponding eval metrics, creating a complete lineage from data to deployed model.

6.5.4 Human Oversight (Art. 14)

Beyond the Human-in-the-Loop mechanisms described above, the system provides two additional transparency layers for human reviewers. First, post-distillation

Method	Accuracy	Natural.	Persuasion	Compliance
Template	–	–	–	–
SHAP+Template	–	–	–	–
3-Agent (ours)	–	–	–	–

Table 7: Human evaluation results (TODO).

LGBM students expose IG-based feature importance rankings per task — a human-readable audit that compliance officers can review without deep learning expertise. Second, CGC gate weights in the PLE teacher quantify each expert's contribution to every prediction, making the expert routing mechanism transparent rather than a black-box ensemble. The 3-agent LLM pipeline (Section 4) further translates these technical attributions into natural-language explanations that compliance officers can directly evaluate.

6.5.5 Accuracy and Robustness (Art. 15)

Article 15 requires that high-risk AI systems achieve "an appropriate level of accuracy, robustness and cybersecurity." The companion paper's ablation study demonstrates *graceful degradation*: removing any single expert reduces performance incrementally rather than causing catastrophic failure, proving that no single component is a critical point of failure. Uncertainty weighting [8] prevents any single task from dominating the multi-task objective, ensuring robust performance across all 14 tasks simultaneously. Drift monitoring (PSI-based) is designed to provide continuous robustness verification once deployed.

6.5.6 Bias Monitoring (Art. 10.2f)

Article 10(2)(f) requires examination of training data for possible biases that may affect health, safety, or fundamental rights. Label distribution validation before training (via `label_stats.json`) ensures class imbalance is documented and addressed. The task-level breakdown in `eval_metrics.json` enables per-segment performance monitoring: auditors can verify that model accuracy does not vary systematically across customer demographics or product categories. The `FairnessMonitor` component (disparate impact, statistical parity difference, equalized odds difference) provides automated bias detection at the granularity required by Article 10(2)(f).

6.5.7 Model Risk Management Lifecycle

The system implements an SR 11-7 aligned five-stage MRM lifecycle: *develop* (teacher training + student distillation), *validate* (independent metric evaluation on held-out data), *approve* (manual sign-off gate; `auto_promote=False` by default), *monitor* (continuous PSI-based drift detection), and *retrain* (automatic re-distillation when drift exceeds threshold). The `ModelCompetitionManager` orchestrates champion-challenger evaluation: when a newly distilled student is produced, it is compared against the current production champion on all target metrics. Promotion requires explicit human approval — the system will never silently replace a production model. When the drift detector fires on consecutive monitoring windows (configurable threshold, default 3 consecutive days), the retrain stage is triggered automatically, producing a new challenger that re-enters the competition cycle. This closed loop ensures that model risk is managed continuously rather than at discrete annual review points, satisfying both SR 11-7 expectations and EU AI Act Art. 9 risk management requirements.

7 Experiments

7.1 Distillation Experiments

7.2 Reason Generation Quality

7.2.1 Human Evaluation Protocol

- **Evaluators**: N financial domain experts (bank product managers, compliance officers).
- **Evaluation criteria**: Accuracy (1-5), Naturalness (1-5), Persuasiveness (1-5), Regulatory fitness (1-5).
- **Comparison**: (A) Template-based, (B) SHAP-based + template, (C) 3-agent pipeline.
- **Method**: Blind evaluation, each evaluator rates 100 recommendation reasons.

7.3 Safety Gate Evaluation

7.4 Serving Performance

7.5 Regulatory Compliance Audit

We evaluate regulatory alignment across three dimensions: *checklist compliance*, *audit trail integrity*, and *fairness metrics*.

Checklist compliance. The system implements 14 regulatory requirements mapped from the Korean Financial Consumer Protection Act (금소법) Articles 17–19, EU AI Act Articles 13–14, and FSS AI Guidelines. Key items include: suitability assessment before recommendation (Art. 19), AI-generated content disclosure (Art. 17),

Check	Precision	Recall	Fallback Rate
Hallucination	–	–	–
Regulatory	–	–	–
Appropriateness	–	–	–
Overall	–	–	–

Table 8: Safety Gate precision and recall (TODO).

Component	Latency	Cost/1K req	Notes
LGBM inference	< 10ms	–	Lambda CPU
IG attribution	< 50ms	–	Lambda CPU
Reason generation	< 200ms	–	Cache hit: < 5ms
Safety Gate	< 50ms	–	Rule-based + LLM
Total	< 300ms	–	Cache hit: < 100ms

Table 9: Serving latency breakdown (Lambda serverless, no GPU).

human oversight mechanism (EU AI Act Art. 14), and opt-out functionality.

Audit trail integrity. Every recommendation reason is logged with: (1) the IG feature attribution vector that produced it, (2) the Safety Gate pass/fail decision and failure reasons if any, (3) the LLM prompt and response pair, (4) a SHA-256 hash chain linking the recommendation to its source model version. The audit log is append-only and stored in a tamper-evident structure suitable for regulatory inspection.

Fairness metrics. We compute Disparate Impact (DI), Statistical Parity Difference (SPD), and Equal Opportunity Difference (EOD) across protected attributes (age group, gender, income tier) for each task. The FairnessMonitor runs as a scheduled batch job and generates alerts when any metric exceeds configurable thresholds.

8 Discussion

8.1 Findings Summary

Three findings emerge from the end-to-end pipeline evaluation.

Finding 1: Tree-based distillation works without temperature scaling. Contrary to the standard Hinton distillation recipe ($T = 3$–20), LGBM students trained with $T = 1$ achieve lower JSD and calibration gap than those trained with $T = 5$. This is consistent with the Soft GBM analysis [3]: tree models learn from the absolute values and ordering of soft labels, not from gradient flow through tail probabilities. The temperature hyperparameter, designed for neural student backpropagation, is unnecessary — and harmful — for split-based learners.

Finding 2: Dual-objective feature selection preserves explanation quality. The α-weighted IG scoring ($\mathrm{IG}_{\mathrm{pred}} + \mathrm{IG}_{\mathrm{explain}}$) retains features that would be dropped by pure predictive selection but are essential for generating business-language recommendation reasons. Features like `hmm_lifecycle_prob_growing` and `synth_monthly_spend` may contribute modestly to AUC but provide the narrative anchors ("growth stage," "spending pattern") that ground LLM-generated explanations.

Finding 3: The Safety Gate is essential, not optional. Template-based fallback without LLM validation produces grammatically correct but occasionally misleading reasons (e.g., citing features not actually influential for the customer). The Safety Gate catches these by cross-referencing generated text against the actual IG attribution vector, reducing hallucination-like errors.

8.2 The Dual Role of Features

A key insight from this work: features serve dual purposes in financial recommendation. Even features with marginal predictive contribution (e.g., TDA topological features may add only $\Delta\mathrm{AUC} = 0.01$) provide irreplaceable context for recommendation reasoning. Internally, TDA persistence captures behavioral shape stability — but the customer never sees "persistent homology" or "Betti numbers." Instead, the `interpretation_registry` reverse-maps this to business language (e.g., "You maintain a stable transaction pattern"), and the LLM agent weaves it into a natural-language reason.

This reframes feature engineering evaluation: the value of a feature is not solely its predictive contribution but also its contribution to the recommendation context

available to the reason generation pipeline. As argued in the companion paper, *what to observe* matters more than *how to model* — features derived from domain-specific questions ("Is their income permanent or transitory?", "Is product adoption spreading like contagion?") enrich the internal reasoning context, enabling more nuanced business-language explanations than any amount of architectural sophistication applied to shallow statistical summaries.

8.3 Practical Deployment Considerations

- **LLM selection**: On-premises LLM vs API (latency, cost, data residency).
- **Reason quality maintenance**: Periodic human review + automated quality scoring.
- **Regulatory updates**: Architecture supports adding new compliance checks without redesign.

8.4 Limitations

- LLM dependency introduces cost, latency, and residual hallucination risk.
- Human evaluation scale limited by expert availability.
- Korean/EU regulatory focus; other jurisdictions not yet analyzed.
- Template fallback quality is inherently limited.

8.5 Future Work

- Real customer A/B testing of reason quality → conversion rate.
- Multi-lingual reason generation (Korean, English, Chinese).
- Automated regulatory update pipeline (regulation change → compliance check update).
- Fine-tuned domain-specific small LLM to replace general-purpose API.

9 Conclusion

We presented a full-chain system that bridges the gap between model prediction and human persuasion in financial product recommendation.

Four key contributions define this work. First, IG-guided knowledge distillation with a dual-objective feature selection preserves both predictive accuracy and explanation material when compressing a complex PLE teacher into lightweight LGBM students. Second, the 3-agent recommendation reason generation pipeline (Feature Selector → Reason Generator → Safety Gate) produces natural-language explanations grounded in business-mapped feature attributions, with role separation enabling independent improvement and audit logging. Third, two operational agents (OpsAgent and AuditAgent) interpret monitoring and compliance outputs in natural language, eliminating dashboard fatigue and enabling regulation-compliant MLOps for small teams without dedicated MLOps staff — extending the architecture to a 5-agent system (3 serving + 2 ops). Fourth, regulatory compliance is embedded by design — Korean FSS guidelines, the EU AI Act, and the Korean AI Basic Act are explicitly mapped to system architecture components, with automated monitoring (drift, fairness, herding) and human-in-the-loop oversight at critical decision points.

The fundamental insight is that features serve a dual role in financial AI: they contribute to prediction *and* to the explanation vocabulary that ultimately persuades customers, empowers relationship managers, and satisfies regulators. This reframes the traditional feature engineering calculus: a feature with marginal AUC contribution but rich business interpretability may be more valuable than a high-AUC feature that generates no meaningful explanation.

The system is designed for deployment on serverless infrastructure (AWS Lambda), achieving sub-100ms serving latency without dedicated GPU servers — matching the operational reality of financial institutions with limited ML engineering resources.

Author Contributions

Seonkyu Jeong (PM / Lead Architect / Data Scientist): Designed the distillation strategy, recommendation reason generation pipeline, and regulatory compliance architecture. Led the overall system design based on domain expertise in financial risk management.

Euncheol Sim: Feature reverse-mapping registry, vector database pipeline, and data ingestion for the serving layer.

Youngchan Kim: Knowledge distillation implementation, model training validation, and mathematical verification of distillation quality.

All authors collaborated through Scrum sprints with rapid feedback cycles.

Funding

This research received no external funding, grants, or institutional infrastructure support. All costs — including AI tools, hardware, mobile connectivity, AWS SageMaker cloud training (Spot instances), S3 storage, and operational expenses — were borne entirely by the first author's personal funds, with development conducted on a single desktop GPU in a resource-constrained environment.

Acknowledgments

The authors thank Yeon-Jin Kim for consistently providing valuable insights on industry trends and marketing perspectives that informed the system's design direction.

The authors express deep gratitude to Euncheol Sim and Youngchan Kim, who dedicated countless nights and weekends to this project with unwavering commitment despite the absence of formal employment or compensation.

Finally, the authors wish to acknowledge the AI tools that made this research possible. Gemini for ideation and design brainstorming, Claude Opus and Sonnet for architecture, implementation, and writing, and Cursor for a seamless development environment. What could have remained an unrealized vision was brought to life through the collaboration of a small team and these tools.

Bibliography

[1] European Banking Authority. 2025. Report on Machine Learning in Internal Models.

[2] European Parliament and Council. 2024. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (AI Act).

[3] Jiawei Feng, Yang Yu, and Zhi-Hua Zhou. 2020. Soft Gradient Boosting Machine. In *arXiv preprint arXiv:2006.04059*, 2020.

[4] Financial Services Commission (Korea). 2024. Financial Sector AI Guidelines.

[5] Milton Friedman. 1957. *A Theory of the Consumption Function*. Princeton University Press.

[6] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. 2015. Distilling the Knowledge in a Neural Network. In *NIPS Deep Learning and Representation Learning Workshop*, 2015.

[7] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. 2017. LightGBM: A Highly Efficient Gradient Boosting Decision Tree. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[8] Alex Kendall, Yarin Gal, and Roberto Cipolla. 2018. Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018.

[9] Scott M Lundberg and Su-In Lee. 2017. A Unified Approach to Interpreting Model Predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.

[10] National Assembly of Korea. 2024. Act on the Development of Artificial Intelligence and Establishment of Trust (AI Basic Act).

[11] Huy Nguyen and others. 2024. Sigmoid Gating is More Sample Efficient than Softmax Gating in Mixture of Experts. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[12] Judea Pearl. 2009. *Causality: Models, Reasoning, and Inference* (2nd ed.). Cambridge University Press.

[13] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. Why Should I Trust You?: Explaining the Predictions of Any Classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.

[14] Amr Salih and others. 2023. Examining the Limitations of Post-hoc Explainability Methods in Finance. *Frontiers in Artificial Intelligence* (2023).

[15] Mukund Sundararajan, Ankur Taly, and Qiqi Yan. 2017. Axiomatic Attribution for Deep Networks. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017.